CS319: Scientific Computing

Projects; A `Vector` class; Linear Equation Solving

Dr Niall Madden

Week 9: 11+13 March, 2026

Slides and examples:
https://www.niallmadden.ie/2526-CS319

2526-CS319

0. Outline

1. Projects!
2. Review of classes
3. Vectors and Matrices
4. A vector `class` (Basic version)
   - C++ "Project"
   - Adding two vectors
5. Systems of equations
   - Algorithms
   - Asides...
6. Iterative methods
7. Jacobi's Method
   - Lab 7
8. Gauss-Seidel Method
   - More history
9. Matrix-based Jacobi
10. Triangular systems
11. Matrix-based Gauss-Seidel
12. Some further details

1. Projects!

Notes for this part are at:
https://www.niallmadden.ie/2526-CS319/2526-CS319-Projects.pdf

2. Review of classes

class

In C++, we define new class with the `class` keyword.
An instance of the class is called an "*object*".
A `class` combines by data and functions (called "methods").

Within a class, code and data may be either

- **Private**: accessible only to another part of that object, or
- **Public**: other parts of the program can access it.

Roughly,

- keep data elements `private`,
- make function elements `public`.

2. Review of classes

The basic syntax for defining a class:

```
class class-name {
private:
    ...     // private functions and variables
public:
    ...     // public functions and variables
};
```

class-name becomes a new object type—one can now declare objects to be of type *class-name*.

This is only a declaration. Therefore,

- functions are not defined, though the prototype is given,
- variables are declared but are not initialised,
- the declaration block is delineated by { and }, and terminated with a semicolon.
- *scope resolution operator*, `::`, used in function definition.

2. Review of classes

- A **Constructor** is a public method of a class, that has the same name as the class. It's return type is not specified explicitly. It is executed whenever a new instance of that class is created.
- A **destructor** is a method that is called on an object whenever it goes out of scope. The name of the destructor is the same as the class, but preceded by a tilde.

3. Vectors and Matrices

This is a course in Scientific Computing.

Many advanced and general problems in Scientific Computing are based around **vectors** and **matrices**. So one of our goals is to implement C++ classes for such structures, along with standard operations such as matrix-vector multiplication.

Along the way, we'll learn about

- operator overloading;
- `friend` functions and the `this` pointer;
- static variables.
- and much more

Our first step will be to study some problems and applications so that, before we design any classes or algorithms, we'll know what we will use them for. These problems include:

1. Basic analysis of matrices, for example with applications to image processing, graphs and networks.
2. Solution of linear systems of equations, for example with applications to data fitting;
3. Estimation of (certain) eigenvalues, for example with applications to Network Science.

Of these problems, probably the most ubiquitous is the solution of (large) systems of simultaneous equations.

That is, we want to solve a linear system of 3 equations in 3 unknowns: *find* x_1, x_2, x_3, *such that*

$$\begin{aligned} 3x_1 + 2x_2 + 4x_3 &= 19 \\ x_1 + 2x_2 + 3x_3 &= 14 \\ 5x_1 + 1x_2 + 6x_3 &= 25 \end{aligned}$$

This can be expressed as a **matrix-vector equation:**

More generally, the linear system of N equations in N unknowns: *find* $x_1, x_2, \ldots, x_N$, *such that*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1N}x_N &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2N}x_N &= b_2 \\ &\vdots \\ a_{N1}x_1 + a_{N2}x_2 + \cdots + a_{NN}x_N &= b_N. \end{aligned}$$

This, as a **matrix-vector equation** is:

$$\begin{pmatrix} a_{11} & a_{12} & \ldots & a_{1N} \\ a_{21} & a_{22} & \ldots & a_{2N} \\ \vdots & & \ddots & \vdots \\ a_{N1} & a_{N2} & \ldots & a_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix}$$

So, to proceed, we need to be able to represent **vectors** and **matrices** in our codes.

4. A vector `class` (Basic version)

Our first focus will be on defining a class of vectors. This version will be quite **basic** (hence the file names). Will be developed later.

Intuitively, we know it needs the following components:

4. A vector class (Basic version) C++ "Project"

Due to the level of detail in the matrix and vector classes, the following example is divided into three source files:

1. `VectorBasic.h`, the header file which contains the class definition. Include this header file in another source file with:
   `#include "VectorBasic.h"`
   Note that this is **not** `<VectorBasic.h>`
2. `VectorBasic.cpp`, which includes the code for the methods in the *`Vector`* class;
3. `OOTestVectorBasic.cpp`, a test stub.

In whatever compiler you are using, you'll need to create a **project** that contains all the files. (Ask Niall for help if needed).

See VectorBasic.h for more details

```
// File: VectorBasic.h (simple version)
// Author: Niall Madden ¡Niall.Madden@UniversityOfGalway.ie¿
// Date: Week 9 of CS319
// What: Header file for vector class
// See also: VectorBasic.cpp and 01TestVectorBasic.cpp
class Vector {
private:
  double *entries;
  unsigned int N;
public:
  Vector(unsigned int Size=2);
  ~Vector(void);

  unsigned int size(void) {return N;};
  double geti(unsigned int i);
  void seti(unsigned int i, double x);

  void print(void);
  double norm(void); // Compute the 2-norm of a vector
  void zero(void); // Set entries of vector to zero.
};
```

VectorBasic.cpp

```
Vector::Vector(unsigned int Size)
{
  N = Size;
  entries = new double[Size];
}

Vector::~Vector()
{
  delete [] entries;
}

void Vector::seti(unsigned int i, double x)
{
  if (i<N)
    entries[i]=x;
  else
    std::cerr << "Vector::seti(): Index out of bounds."
              << std::endl;
}
```

VectorBasic.cpp continued

```
double Vector::geti(unsigned int i)
{
  if (i<N)
    return(entries[i]);
  else {
    std::cerr << "Vector::geti(): Index out of bounds."
              << std::endl;
    return(0);
  }
}

void Vector::print(void)
{
  for (unsigned int i=0; i<N; i++)
    std::cout << "[" << entries[i] << "]" << std::endl;
}
```

VectorBasic.cpp continued

```
double Vector::norm(void)
{
  double x=0;
  for (unsigned int i=0; i<N; i++)
    x+=entries[i]*entries[i];
  return (sqrt(x));
}

void Vector::zero(void)
{
  for (unsigned int i=0; i<N; i++)
    entries[i]=0;
}
```

4. A vector class (Basic version) Adding two vectors

Here is a simple implementation of a function that computes $\mathsf{c} = \alpha\mathsf{a} + \beta\mathsf{b}$

See `00TestVectorBasic.cpp` for more details

```
// c = alpha*a + beta*b where a,b are vectors; alpha, beta are scalars
void VecAdd (vector &c, vector &a, vector &b,
        double alpha, double beta)
{
  unsigned int N;
  N = a.size();

  if ( (N != b.size()) )
    std::cerr << "dimension mismatch in VecAdd " << std::endl;
  else
  {
    for (unsigned int i=0; i<N; i++)
        c.seti(i, alpha*a.geti(i)+beta*b.geti(i) );
  }
}
```

5. Systems of equations

As mentioned earlier, probably the most ubiquitous problem in scientific computing is the solution of (large) systems of simultaneous equations.

Here is an example (previously seen) where we want to solve a linear system of 3 equations in 3 unknowns: *find* x_1, x_2, x_3, *such that*

$$\begin{aligned} 3x_1 + 2x_2 + 4x_3 &= 19 \\ x_1 + 2x_2 + 3x_3 &= 14 \\ 5x_1 + x_2 + 6x_3 &= 25 \end{aligned}$$

This can also be expressed as a **matrix-vector equation**; $Ax = b$, where

$$\underbrace{\begin{pmatrix} 3 & 2 & 4 \\ 1 & 2 & 3 \\ 5 & 1 & 6 \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_{x} = \underbrace{\begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}}_{b}$$

5. Systems of equations

More generally, the linear system of N equations in N unknowns: *find* $x_1, x_2, \ldots, x_N$, *such that*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1N}x_N &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2N}x_N &= b_2 \\ &\vdots \\ a_{N1}x_1 + a_{N2}x_2 + \cdots + a_{NN}x_N &= b_N. \end{aligned}$$

This, as a **matrix-vector equation** is:

$$\begin{pmatrix} a_{11} & a_{12} & \ldots & a_{1N} \\ a_{21} & a_{22} & \ldots & a_{2N} \\ \vdots & & \ddots & \vdots \\ a_{N1} & a_{N2} & \ldots & a_{NN} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_N \end{pmatrix}$$

5. Systems of equations

So, eventually, we need to be able to represent **vectors** and **matrices** as classes in our codes.

However, in the short term, we'll make do with just 1D and 2D arrays.

There are several classic approaches for solving linear systems:

1. Gaussian Elimination, often expressed as LU-factorisation
2. Cholesky factorisation (which is LL^T for symmetric positive definite matrices)
3. Stationary Iterative schemes such as **Jacobi's method**, **Gauss-Seidel** and Successive Over Relaxation (SOR);
4. Krylov subspace methods, of which Conjugate Gradients is the best known;
5. Enhancements of the Methods 3 and 4, using preconditioning with, for example, MultiGrid and Incomplete LU-factorisation.
6. But never, *ever* solving $Ax = b$ by computing A^{-1}, unless $N = 2$ or $N = 3$.

Of the approaches listed above, Jacobi's is by far the simplest to implement, and so is the one we will study first.

Some facts about the previously mentioned methods:

- What we call "Gaussian Elimination" was known in Chinese mathematics for at least 2,000 years.
- It was known in European mathematics since the 1500's, but it is believed that the method was so obvious, nobody bother writing it down!
- Eventually, Newton did in his personal notes, later published as *Arithmetica Universalis* in 1707.
- Gauss devised some notation for the symmetric case. But the name "Gaussian Elimination" came from some textbooks published in the 1950s, which (mis)-attributed the method to him.
- The modern version (LU-factorisation) is understood as a matrix-based method, since the work of Tadeusz Banachiewicz in the late 1930s. (Turing and von Neumann were pivotal too).

However, Gaussian Elimination (GE), is an example of what is called a **direct method**. That means, we arrive at a solution after a fixed number of steps, and that the solution is exactly correct (if we can neglect round-off error...)

However, although it is very efficient, GE has many technical difficulties.

- In many applications, the system matrix has very structured, and that can be exploited to great effect. For example, A may be *sparse* (come back next week to learn what that means). But Gaussian Elimination may destroy the sparseness. As a result, it can be very memory intensive.
- It can be *ill-conditioned*, meaning that small errors (e.g., due to the use of floating point numbers) can be greatly magnified.

So we are not going to study it any more in CS319! Instead, we'll consider **iterative methods**

6. Iterative methods

The alternative to **direct methods** are **iterative methods**. These are algorithms which take an initial guess for the solution, and then refine it, again and again, to get better and better approximations of the solution.

That is, they compute a sequence

$$x^{(0)}, x^{(1)}, x^{(2)}, \ldots,$$

where, if everything works correctly, $\lim_{n\to\infty} \|x - x^{(n)}\| = 0$.

That is, if we take an infinite number of iterations (and have exact arithmetic) we get the true solution.

However, we don't try to take an infinite number of iterations, for two reasons:

(a) We don't have a infinite amount of (CPU) time at our disposal;

(b) It is not necessary: modelling error or numerical error are not zero.

6. Iterative methods

In a sense, such iterative methods are similar to Newton's method we worked on in Labs 3+4:

- We take an initial guess;
- We repeatedly refine that;
- We stop when some measure of the error is small enough.

However, there are some differences too: in particular, Newton's method may fail if the initial guess is not sufficiently close to the actual solution. The methods we'll study will converge for *any* initial guess (though they may take longer if the initial guess is not particularly good).

However, we can easily evaluate how good an estimate for the solution is, by computing the **residual**: if $x^{(k)}$ is an estimate for the solution to $Ax = b$, the *residual* is $\|b - Ax^{(k)}\|$.

7. Jacobi's Method

Jacobi's Method is an example of an *iterative method*. It can be derived as follows:

7. Jacobi's Method

In summary, **Jacobi's method** is: *choose* $\mathrm{x}^{(0)}$ *and set*

$$
\begin{aligned}
x_1^{(k+1)} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1N}x_N^{(k)}) \\
x_2^{(k+1)} &= \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \cdots - a_{2N}x_N^{(k)}) \\
&\vdots \\
x_N^{(k+1)} &= \frac{1}{a_{NN}}(b_N - a_{N,1}x_1^{(k)} - \cdots - a_{N,N-1}x_{N-1}^{(k)})
\end{aligned}
$$

This can be programmed with two (or so) nested `for` loops.

7. Jacobi's Method

Lab 7

An implementation is (or will be) given in Lab 7: https://www.niallmadden.ie/2526-CS319/lab7/Jacobi-Lab7.cpp

It works as follows:

- The two-dimensional array `A` stores the coefficients for the left-hand side.
- The one-dimensional arrays `x` and `b` stores the true solution and left-hand side, respectively.
- The one-dimensional arrays `xk` and `xk1` represent the vectors $x^{(k)}$ and $x^{(k+1)}$.

- It sets `A` and `b` to represent the problem

$$9x_1 + 3x_2 + 3x_3 = 15 \quad (1)$$
$$3x_1 + 9x_2 + 3x_3 = 15 \quad (2)$$
$$3x_1 + 3x_2 + 9x_3 = 15 \quad (3)$$

  The true solution is $x_1 = x_2 = x_3 = 1$.
- Five iterations of the Jacobi method are taken.
- The estimated solution after five iterations is outputted.

7. Jacobi's Method

In that lab you are asked to make various improvements...

8. Gauss-Seidel Method

Jacobi's method is not particularly efficient. Heuristically, you argue that it could be improved as follows. In Jacobi's method, we compute $x_1^{(k+1)}$ from

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1N}x_N^{(k)})$$

We expect that it is a better estimate for x_1 than $x_1^{(k)}$.

Next we compute

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \cdots - a_{2N}x_N^{(k)})$$

However, here we used the "old" value $x_1^{(k)}$ even though we already know the new, improved $x_1^{(k+1)}$. That is, we could use

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)} - \cdots - a_{2N}x_N^{(k)})$$

8. Gauss-Seidel Method

See notes on board

8. Gauss-Seidel Method

More generally, in Jacobi's method we set

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j=1, j\neq i}^{N} a_{ij} x_j^{(k)}\Big).$$

The Gauss-Seidel method uses

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\Big(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{N} a_{ij} x_j^{(k)}\Big).$$

In Lab 7, you are asked to implement this method, and verify that it is more efficient than the Jacobi method, in the sense that fewer iterations are required to achieve the same level of accuracy.

The historical links between the Jacobi and GS methods are a little vague, but a few things are known:

1. GS converges more efficiently, and also is actually easier to code. And requires less memory... (why?)
2. Both methods are very **tolerant of errors**. That is, if you are implementing one of the methods, and you make a arithmetic mistake, the method will still, eventually, yield a sensible solution (unlike GE...)
3. GS predates Jacobi, and, in some settings, Jacobi is considered an improvement upon GS.

Point 2+3 may be clearer if you recall that these methods were designed to be implemented by human "computers", often working in parallel...

9. Matrix-based Jacobi

To recap, we are solving a linear system of N equations in N unknowns: *find* $x_1, x_2, \ldots, x_N$, *such that*

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1N}x_N &= b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2N}x_N &= b_2 \\ &\vdots \\ a_{N1}x_1 + a_{N2}x_2 + \cdots + a_{NN}x_N &= b_N. \end{aligned}$$

We express this as a matrix-vector equation: *Find* x *such that*

$$A\mathrm{x} = \mathrm{b},$$

where A *is a* $N \times N$ *matrix, and* b *and* x *are (column) vectors with* N *entries.*

9. Matrix-based Jacobi

Then **Jacobi's method** is: choose $\mathrm{x}^{(0)}$ and set

$$
\begin{aligned}
x_1^{(k+1)} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)} - \cdots - a_{1N}x_N^{(k)}) \\
x_2^{(k+1)} &= \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)} - \cdots - a_{2N}x_N^{(k)}) \\
&\vdots \\
x_N^{(k+1)} &= \frac{1}{a_{NN}}(b_N - a_{N1}x_1^{(k)} - \cdots - a_{N,N-1}x_{N-1}^{(k)})
\end{aligned}
$$

There is also a matrix-version of this iteration. We set D and T to be the matrices

$$
d_{ij} = \begin{cases} a_{ii} & i = j \\ 0 & \text{otherwise.} \end{cases} \qquad t_{ij} = \begin{cases} 0 & i = j \\ -a_{ij} & \text{otherwise.} \end{cases}
$$

9. Matrix-based Jacobi

So $A = D - T$. Then *Jacobi's method* can be written neatly in matrix form:

$$x^{(k+1)} = D^{-1}(b + Tx^{(k)}). \tag{4}$$

. . . . . .

The significance for us is:

- We should implement C++ classes for vectors and matrices;
- We need to "overload" the assignment operator `=` for `Vector`s.
- We should overload the `+` and `-` operators for `Vector` addition and subtraction.
- We need to overload the `Matrix`-`Vector` multiplication operator.

9. Matrix-based Jacobi

If we do all that, we can implement the Jacobi method "just"[1] a few lines:

```
while ( (Rnorm > TOL) && (count<max_its))
{
  count++;
  xk1 = Dinv*(b+T*xk); // set x = inverse(D)*(b+T*x)
  R = A*xk-b;
  Rnorm = R.norm();
  xk = xk1;
}
```

Of course, this depends on already having defined the matrices `T` and `Dinv` first.

[1] Investing time and energy in developing data structures and methods which then allow us to succinctly implement algorithms is a touch-stone of scientific computing.

10. Triangular systems

Next we want to consider a matrix-based GS method. First we consider triangular systems.

Some systems of equations are very easier to solve than others. Suppose the system is $L\mathrm{x} = \mathrm{b}$, but L is a lower triangular matrix. The associated system of equations looks like this:

$$\begin{aligned}
l_{11}x_1 &= b_1 \\
l_{21}x_1 + l_{22}x_2 &= b_2 \\
l_{31}x_1 + l_{32}x_2 + l_{33}x_3 &= b_2 \\
&\vdots \\
l_{N1}x_1 + l_{N2}x_2 + \cdots + l_{NN}x_N &= b_N.
\end{aligned}$$

To solve this,

- first set $x_1 = b_1/l_{11}$.

10. Triangular systems

- Now substitute this into the second equation to get $x_2 = (b_2 - l_{21}x_1)/l_{22}$.
- Next we use $x_3 = (b_3 - l_{31}x_1 - l_{32}x_2)/l_{33}$,
- and so on.

In fact, this is quite like Jacobi's method, except we don't have to iterate...

Suppose we code an implementation for a (lower) triangular matrix, `L`. Then, since `L*x= b`, it is reasonable to write `x=b/L`, where the "forward slash" operator is implementing back-substitution as just described.

11. Matrix-based Gauss-Seidel

Recall that the **Gauss-Seidel method** is choose $\mathrm{x}^{(0)}$ and set

$$
\begin{aligned}
x_1^{(k+1)} &= \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k+1)} - a_{13}x_3^{(k)} - \cdots - a_{1N}x_N^{(k)}) \\
x_2^{(k+1)} &= \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k+1)} - \cdots - a_{2N}x_N^{(k)}) \\
&\vdots \\
x_N^{(k+1)} &= \frac{1}{a_{NN}}(b_N - a_{N1}x_1^{(k+1)} - \cdots - a_{N,N-1}x_{N-1}^{(k+1)})
\end{aligned}
$$

In the same way as we did for Jacobi's method, we can write this in a succinct matrix-vector form: we set L and U to be the matrices

$$
l_{ij} = \begin{cases} a_{ij} & i \geq j \\ 0 & \text{otherwise.} \end{cases} \qquad u_{ij} = \begin{cases} 0 & i \geq j \\ -a_{ij} & \text{otherwise.} \end{cases}
$$

11. Matrix-based Gauss-Seidel

So $A = L - U$. Then the *Gauss-Seidel method* can be written as

$$Lx^{(k+1)} = b + Ux^{(k)}. \tag{5}$$

Note that this involves solving a linear system where L is the coefficient matrix.

12. Some further details

- Writing $A = L - U$ is called a **regular splitting** of A.
- **Important** For either Jacobi or GS to converge, it is sufficient (but not necessary) for A to be "SDD": strictly diagonally dominant (meaning...)
- Being "SDD" just ensures that the spectral radius of the iteration matrix a is less than 1.